documentation (exercise 5)

Roberto Sonzini Gobbi: 12505287

For the assignment I followed the honor code:

- Rule 1: Don't look at solutions or code of other students; everything you submit should be your own work
- Rule 2: Don't share your solution code with others; however, discussing ideas or general strategies is fine and encouraged
- Rule 3: If you use code from somewhere you need to cite the source directly in your code (e.g., StackOverflow). If the source is a generative AI tool (e.g., ChatGPT) provide a link to the specific Chat. I know that I have full responsibility for what I submit. I made sure the assignment allows the teaching staff to evaluate which competences I have acquired. I am aware that plagiarism is prohibited and may lead to failure of this course.

1 Question 1

The final results of the object recognition process are presented below, obtained using two slightly different approaches. In the first method, SIFT is applied to the test image as a whole: all objects detected through clustering are represented within a single composite scene. In the second method, the image is first segmented, and SIFT is applied object by object, allowing for a more detailed and localized representation of the individual elements present in the scene.



(a) Image000 (b) Image001 (c) Image002 (d) Image003

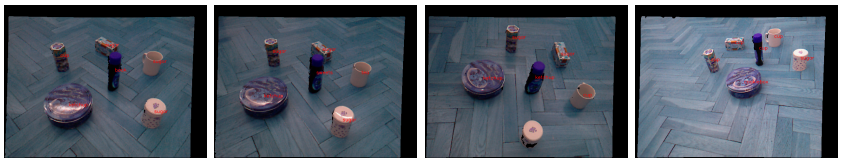


(e) Image004 (f) Image005 (g) Image006



(h) Image007 (i) Image008 (j) Image009

Figure 1: Object recognition with first method



(a) Image000 (b) Image001 (c) Image002 (d) Image003



(e) Image004 (f) Image005 (g) Image006



(h) Image007 (i) Image008 (j) Image009

Figure 2: Object recognition with second method

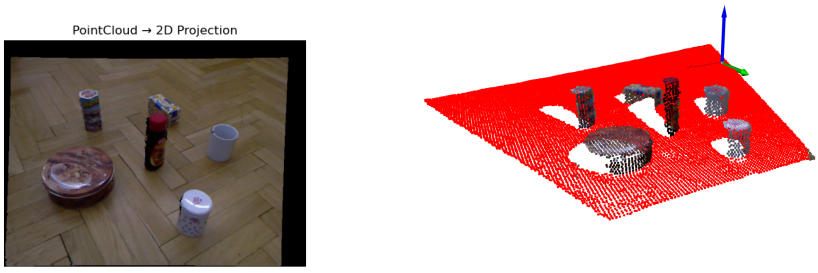
2 Question 2

Below, I present my approach to the problem, explaining the intentions and ideas behind each step to achieve the final result.

2.1 Step 1: Upload of the original image and of the cloud-points

In this first simple step, I loaded the point clouds of the test image (the one on which object recognition will later be performed) and projected them into 2D using the camera parameters. This allowed me to obtain a 2D test image, which will later be used to generate other images, such as those required for SIFT with isolated objects.

In the same step, I also performed downsampling of the 3D points. This was done to enable faster computation of clusters and, more generally, to better manage the 3D point cloud, which, due to the high number of points, could otherwise slow down the entire program.



(a) Projected scene in 2D

(b) Downsampled pointcloud with detected ground

Figure 3

2.2 Step 2: Clustering

To perform object recognition later, a clustering algorithm was implemented. In particular, DBSCAN was chosen, as previous experiments showed it to be one of the best performers for this type of task.

Before running the clustering algorithm, RANSAC was applied to retain only the elements/objects resting on the plane as points present in the scene.

In some cases, DBSCAN can present challenges due to the way the algorithm works. A known issue arises when clusters are very close to each other; in such cases, DBSCAN may detect multiple objects as a single cluster, which can lead to obvious problems in subsequent recognition.

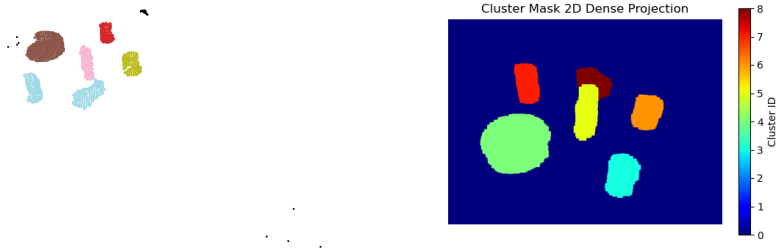
One potential solution could be to use multiple clustering algorithms to detect objects and see if, for a single cluster detected by DBSCAN, the others identify a division. If there is a majority vote supporting this division, the cluster could then be split.

Another possible approach is to run a second clustering algorithm, or even another DBSCAN with different parameters, on clusters that are suspected of containing multiple objects. As a decision criterion, we could use a threshold on the number of points within a cluster, since we would expect that when DBSCAN merges multiple objects into one cluster, it contains significantly more points. (However, this criterion does not necessarily always work, as several small objects close together may not exceed the threshold.)

2.3 Step 3: Cluster filtering

Since, in some cases, after removing the supporting plane and due to noise in the point cloud, there may be clusters of points that do not correspond to real objects in the scene, a statistical filter was implemented to remove these small clusters.

Various approaches could be used, but the simplest method, which already yielded satisfactory results, is based on the median and standard deviation. Specifically, the method assumes that "fake" clusters are those with a number of points below a certain threshold, calculated as $median - k \cdot std$, where k is a chosen hyperparameter.



(a) Detected clusters with DBSCAN (b) 2D projection of clusters

Figure 4

2.4 Step 4: Create a 2D map of clusters and the fragmented image

To create the fragmented image on which SIFT would later operate, a 2D mask of the clusters was generated. Specifically, the clusters were projected into 2D so that each pixel in the reference image could be assigned to a detected cluster.

This was done in order to obtain a final image in which the scene contained either a single object (one cluster) or all the objects only.

After creating the mask, the fragmented image was then generated.



(a) Object 1



(b) Object 2



(c) Object 3



(d) Object 4



(e) Object 5



(f) Object 6



(g) All objects in a scene

Figure 5: Visualization of the detected objects

2.5 Step 5: SIFT for Object Recognition

The final step involved using SIFT, specifically its descriptors, to assign a label to each object/cluster present in the image.

With SIFT, it is possible to compute descriptors from an image and match them with descriptors from another image to determine whether they correspond to the same object. A major advantage of SIFT is its invariance to rotations,

which allows it to find correspondences between objects even if the same object is not in the same position.

The richer and more informative the descriptors are, the more accurate the matches between images, leading to better quality and, in terms of object recognition, higher accuracy of the results.

In this project, SIFT was implemented in two ways: comparing the training images (single objects extracted from scenes) with either a scene containing all objects or with separate scenes containing individual objects.

The first approach was easier to implement and required less computation, while the second approach proved to yield higher accuracy.

The reason for this behavior is that, in the second approach, fewer keypoints are detected outside the object of interest, allowing a more precise evaluation of how similar a given object is to a training image.

For the first approach (single fragmented image), once keypoints and matches were computed, each keypoint falling within a cluster was given a vote for the class of the corresponding training image. The vote was normalized by the number of keypoints in the training image. This normalization is necessary because larger objects generally have more keypoints, which could bias the comparison in favor of objects with more keypoints.

In the second approach, the count was computed for each cluster/object using the training images, with each training image voting only for its corresponding class. Since this was a direct comparison, only the maximum number of matches for each class was retained. The same normalization as above was applied for the same reasons.

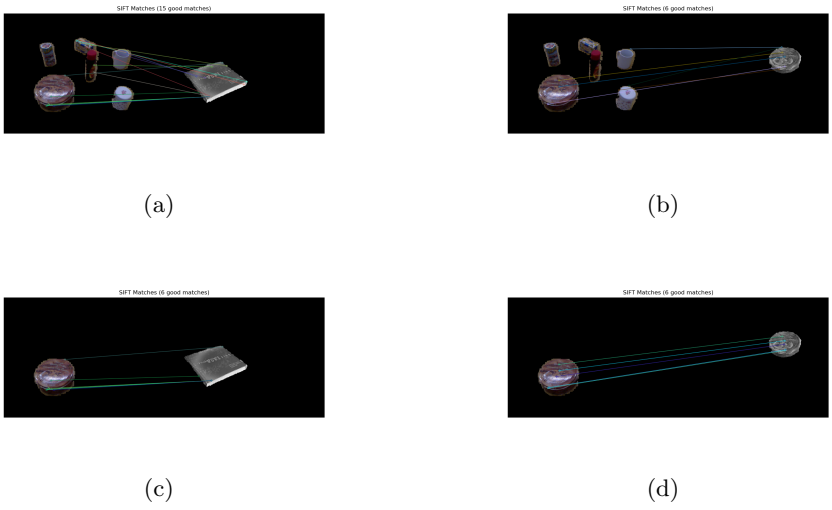


Figure 6: SIFT matching examples

2.6 Step 6: Presentation of the Results

Using the previously computed mask and the original 2D image, the predicted labels were overlaid on the image. Additionally, approximate metrics such as accuracy, precision, and recall were computed. The results will be explained and presented in more detail in the following section.

3 Question 3

In order to evaluate the model, the standard metrics of accuracy, precision, and recall were considered. It is important to note that these metrics were computed in an approximate manner, as the ground truth does not provide a direct mapping between each detected cluster and the corresponding object.

Consequently, the reported values should be interpreted as a preliminary qualitative assessment of the model's performance. Additional verification and more rigorous evaluation would be required to reliably quantify its true accuracy and robustness.

Table 1: Average recognition metrics for the two SIFT-based approaches

Method	**Accuracy**	**Precision**	**Recall**
Whole image approach	0.667	0.935	0.693
Segmented image approach	0.676	0.955	0.693

Table 2: Execution time comparison between the two recognition approaches

Method	**Time [s]**
Whole image approach	160.596
Segmented image approach	155.149

From Tables 1 and 2, we can observe that the two approaches are overall very similar, as expected. The method that applies a mask to isolate individual objects shows slightly higher accuracy and precision. However, this difference is not substantial enough to consider one method clearly superior to the other.

Execution time was also compared, revealing a similar trend. The segmented-object approach exhibits a slightly lower execution time. This is likely because computing SIFT on individual objects is lighter, and although more intermediate steps are required to reach the final result, the overall computation is faster. Nevertheless, the performance gain is not significant, and greater improvements could likely be achieved with a larger training set.

Regarding step-wise execution times, the results are reported in Table 3. As expected, the most time-consuming stage is the SIFT matching, which confirms the observations discussed above.

Table 3: Execution time of each pipeline step

Pipeline step	**Time [s]**
Load, 2D projection and downsampling	5.970
Clustering	3.478
Mask creation	1.422
Object extraction	3.028
Training data loading	2.385
Recognition (SIFT matching)	4.233
Total	**20.518**

Looking at the results obtained, it is evident that the object identification threshold in the scene was not satisfactorily achieved. Consequently, more advanced techniques or improvements should be considered in order to obtain better performance. One potential approach would be the training of a Convolutional Neural Network (CNN), which typically demonstrates high performance in tasks of this nature.

3.1 Attempted Solutions for Improved Results (Extra)

In contemporary contexts, SIFT descriptors are often superseded by alternative approaches, such as neural networks. For this project, I explored two strategies aimed at improving classification performance: fine-tuning a pre-trained model (ResNet18) and constructing a small convolutional network from scratch.

The first approach is particularly appealing because it leverages a model that has already been trained on large datasets and on tasks that are generally similar to the one of interest. Fine-tuning involves providing our labeled training examples and retraining at least the final classification layer, although it is possible to fine-tune multiple layers to further adapt the model.

The second approach involves designing a compact convolutional network. This strategy was considered because, given the limited size of the dataset, a larger model with many parameters could struggle to capture the underlying structure of the data, resulting in a higher number of misclassifications.

In both cases, data augmentation was essential to increase the effective size of the training set, which was extremely small. Data augmentation was implemented using PyTorch, which allows for real-time image transformations during training. Transformations such as zooming in and out, translations, and rotations were applied, as they closely reflect the variations expected in the test images.

Unfortunately, the training outcomes in both cases were unsatisfactory, leading the networks to collapse into predicting a single class. This behavior is likely a consequence of the dataset's limited size, which prevented the models from generalizing effectively.

Although fine-tuning is a widely used approach, I was therefore unable to integrate the models into the prediction pipeline. By design, such integration

would have been straightforward, as the program supports object-by-object recognition.

3.2 Other Potential Solutions for Improved Results

Although not implemented, several additional strategies could be considered to enhance performance.

One option is to continue using neural networks but leveraging a concept similar to that used with SIFT. Specifically, a model could be trained to output a similarity score between two input images. By maintaining an array of similarity scores, the class with the highest probability could then be selected.

Another approach involves developing more informative SIFT descriptors, which could improve matching accuracy and reduce misclassifications. In this context, it would also be possible to select a subset of descriptor features and apply explainable AI techniques, such as SHAP, to determine which features are most relevant for classification.